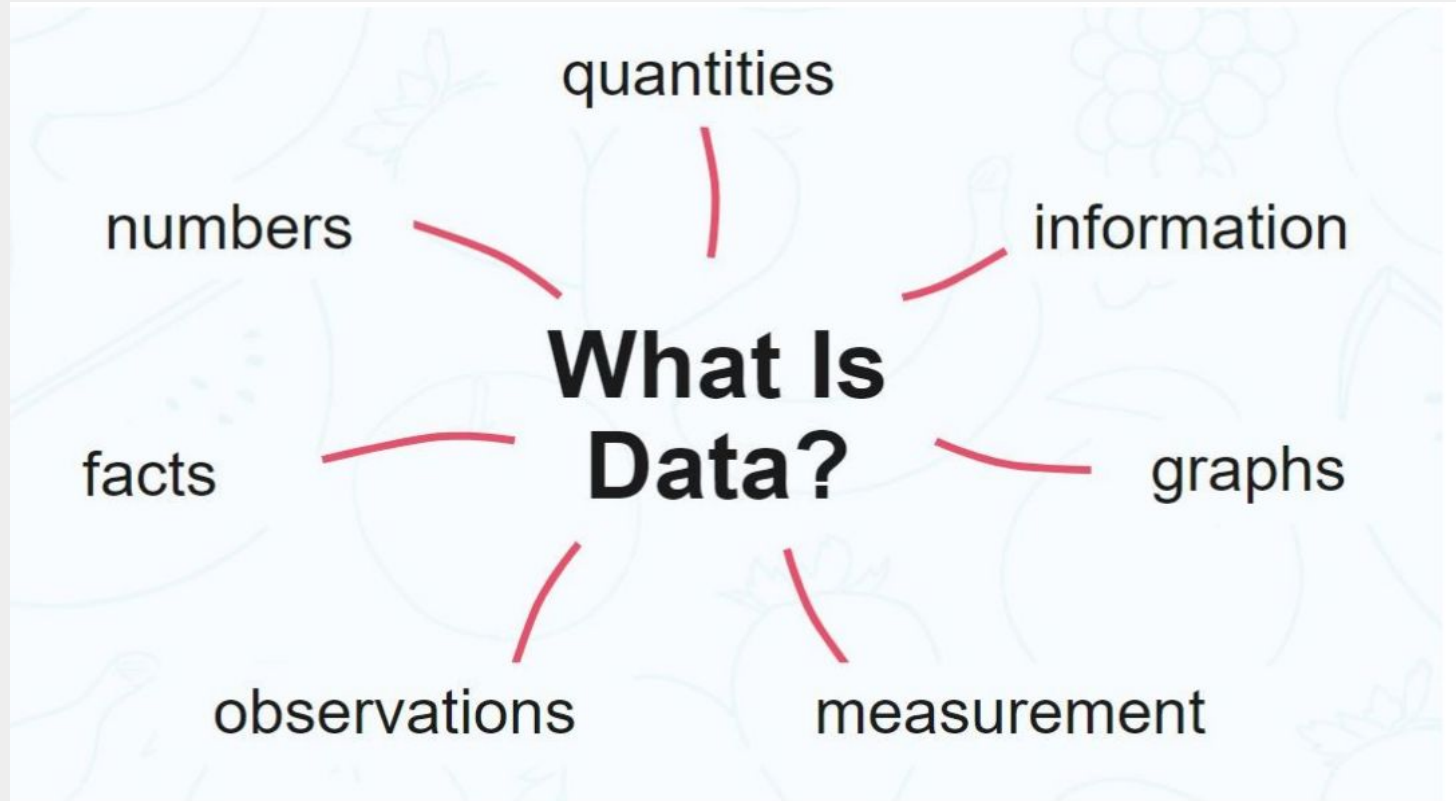


DATABASES AND SQL

What Is Data?



What is Data?

- Data is a collection of a distinct small unit of information. It can be used in a variety of forms like text, numbers, media, bytes, etc. it can be stored in pieces of paper or electronic memory, etc.
- Word 'Data' is originated from the word 'datum' that means 'single piece of information.' It is plural of the word datum.
- In computing, Data is information that can be translated into a form for efficient movement and processing. Data is interchangeable.

The Problem?

- Imagine you're running a college. You have:
 - 2000+ students
 - 50+ faculty members
 - 100+ courses
 - Thousands of enrollment records
 - Marks, attendance, payments, etc.
- **The Problem:** How do you store, organize, and retrieve this data efficiently?
- **Old Way:** Excel sheets, scattered files, manual searching = chaos and errors
- **New Way:** Database Management System (DBMS) = organized, fast, secure, scalable

Data vs. Information – The Core Difference

- Data:
 - Raw facts and figures
 - Example: "25", "Pune", "2023-12-01"
 - No context, no meaning
 - Just symbols and numbers
- Information:
 - Processed data with context and meaning
 - Example: "Student ID 25 is from Pune and enrolled on 2023-12-01"
 - Useful for decision-making
 - Derived from organizing and analyzing data

Example

- Data: 85, 92, 78, 88, 95
- Information: The class average is 87.6, with a top score of 95
- Insight: Most students are performing well, only 1 student needs intervention

What is Database?

- Databases (and SQL) are tools that can be used to interact with, store, and manage information. Although the tools we're using in this course are new, a database is an age-old idea.
- Look at this diagram from a few thousand years ago. It has rows and columns, and seems to contain stipends for workers at a temple. One could call this diagram a table, or even a spreadsheet.

Contents: The payment of grain and dates as temple stipends for twelve months.

	[Nisannu]	Abzu	Sinshu	Dāzu	Abu	Ušutu	naphar	Tashritu	Arah-shamma	Kislimu	Tebitu	Shabatu	Adaru	naphar sk'um	naphar	a-wi-lu-tum	MU-BI-im
							sha i-na gāt mīfu nabi mah-rum							sha gāt libbi abulli i-na libbi she'i sha Zarat- IM ki u suluppū mahrum			
																KAL	mSin-da-ma-qu, "overseer."
	72ga	72ga	72ga	72ga	72ga	72ga	2gur 72ga	72ga	72ga	72ga	72ga	72ga	72ga	2gur 72ga	4gur 144ga	KAL	mSin-mu-shab-shi, "temple servant."
	72ga	72ga	72ga	72ga	72ga	72ga	2gur 72ga	72ga	72ga	72ga	72ga	72ga	72ga	2gur 72ga	4gur 144ga	KAL	mIp-pa-e-a ù(ditto)
5	72ga	72ga	72ga	72ga	72ga	72ga	2gur 72ga	72ga	72ga	72ga	72ga	72ga	72ga	2gur 72ga	4gur 144	KAL	mIdinanni-Shamash, "keeper."
	30ga	30ga	30ga	30ga	30ga	30ga	1 gur	30ga	30ga	30ga	30ga	30ga	30ga	1 gur	2 gur	SAL or i	Tam-bi-Da-du, "his wife."
	24ga	24ga	24ga	24ga	24ga	24ga	141 ga	24ga	24ga	24ga	24ga	24ga	24ga	144 gur	1gur 108ga	SAL-TUR	Da-ti-lu-sha, "his daughter", "seeress."
	18ga	18ga	18ga	18ga	18ga	18ga	108 ga									KAL-TUR	mArdu-Nusku, "his son," ultu Tashritu harrānu.
	12ga	12ga	12ga	12ga	12ga	12ga	72 ga	12ga	12ga	12ga	12ga	12ga	12ga	72 ga	144 ga	KAL-TUR-TUR	mNusku-ki-na-u-gur, "his [grand] son."
10	6ga	6ga	6ga	6ga	6ga	6ga	36 ga	6ga	6ga	6ga	6ga	6ga	6ga	36 ga	72 ga	TUR-GAB	mGab-mar-la-ash, "his son."
	72ga	72ga	72ga	72ga	72ga	72ga	2gur 72ga	72ga	72ga	72ga	72ga	72ga	72ga	2gur 72ga	4gur 144ga	KAL	mA-na-dShe-mi-i-at-kal, "grinder."
	48ga	48ga	48ga	48ga	48ga	48ga	1gur 108ga	48ga	48ga	48ga	48ga	48ga	48ga	1gur 108ga	3gur 36ga	SAL or i	Ish-tar-be-li-uš-ri.
																KAL-TUR	mUshab-shi-uz-ni-a-na-ili, "her son," harrānu.
	24ga	24ga	24ga	24ga	24ga	24ga	144 ga	24ga	24ga	24ga	24ga	24ga	24ga	144 ga	1gur 108ga	KAL-TUR-TUR	mDu-uk-ki-in-ilu, bSHI(?) "her [grand] son."
15	30ga	30ga	30ga	30ga	30ga	30ga	1 gur	30ga	30ga	30ga	30ga	30ga	30ga	1 gur	2 gur	SAL-TUR	Ba-su-un-du, "her daughter," "seeress."
	12ga	12ga	12ga	12ga	12ga	12ga	72 ga	12ga	12ga	12ga	12ga	12ga	12ga	72 ga	144 ga	SAL-TUR-GAB	Hu-la-la-tum, "her daughter."
	6ga	6ga	6ga	6ga	6ga	6ga	36 ga	6ga	6ga	6ga	6ga	6ga	6ga	36ga	72 ga	TUR-GAB	I-na-rish-Marduk-di-nu, "her son."
	48ga	48ga	48ga	48ga	48ga	48ga	1gur 108ga	48ga	48ga	48ga	48ga	48ga	48ga	1gur 108ga	3gur 36ga	SAL or i	Bilut-balātu-ti(rish(-ish))
	18ga	18ga	18ga	18ga	18ga	18ga	108 ga	18ga	18ga	18ga	18ga	18ga	18ga	108 ga	1gur 36ga	KAL-TUR-TUR	mLul-la-mar-Nusku, "her son," "weaver."
20	12ga	12ga	12ga	12ga	12ga	12ga	72 ga	12ga	12ga	12ga	12ga	12ga	12ga	72 ga	144 ga	SAL-TUR-GAB	Rabi-sha-dIsh-ha-ra, "her daughter."
	6ga	6ga	6ga	6ga	6ga	6ga	36 ga	6ga	6ga	6ga	6ga	6ga	6ga	36 ga	72 ga	SAL-TUR-GAB	Di-mi-ili-lu-mar, "her daughter."
	48ga	48ga	48ga	48ga	48ga	48ga	1gur 108ga	48ga	48ga	48ga	48ga	48ga	48ga	1gur 108ga	3gur 36ga	SAL or i	Mi-sha-ri-tum.
	48ga	48ga	48ga	48ga	48ga	48ga	1gur 108ga	48ga	48ga	48ga	48ga	48ga	48ga	1gur 108ga	3gur 36ga	SAL or i	I-na-Ak-ka-di-rab-bal.

What is Database?

- In today's information age, we can store our tables using software like Google Sheets instead of paper or stone tablets. However, in this course we will talk about databases and not spreadsheets.
- A database is an organized collection of data, so that it can be easily accessed and managed.
- You can organize data into tables, rows, columns, and index it to make it easier to find relevant information.

What is Database?

- Three reasons to move beyond spreadsheets to databases are
 - Scale: Databases can store not just items numbering to tens of thousands but even millions and billions.
 - Update Capacity: Databases are able to handle multiple updates of data in a second.
 - Speed: Databases allow faster look-up of information. This is because databases provide us with access to different algorithms to retrieve information. In contrast, spreadsheets that merely allow the use of Ctrl+F or Cmd+F to go through hits one at a time.

File System vs. DBMS

- File System (Traditional Approach)
- How it works:
 - Store data in files (.txt, .csv, .xlsx, etc.)
 - Each file is independent
 - Search using file system utilities

File System vs. DBMS

- Limitations:
 - Redundancy: Same data stored in multiple files → huge storage waste + inconsistency
 - Example: Student "Raj" is in marks.xlsx, attendance.csv, fees.xlsx with different phone numbers!
 - Difficulty in retrieving related data: If you want "all students from Computer Dept with marks > 80", you need to:
 - Open department file
 - Cross-check with marks file
 - Manual filtering
 - Prone to human error
 - Poor Security: You can't control who sees what Anyone with file access can see everything

File System vs. DBMS

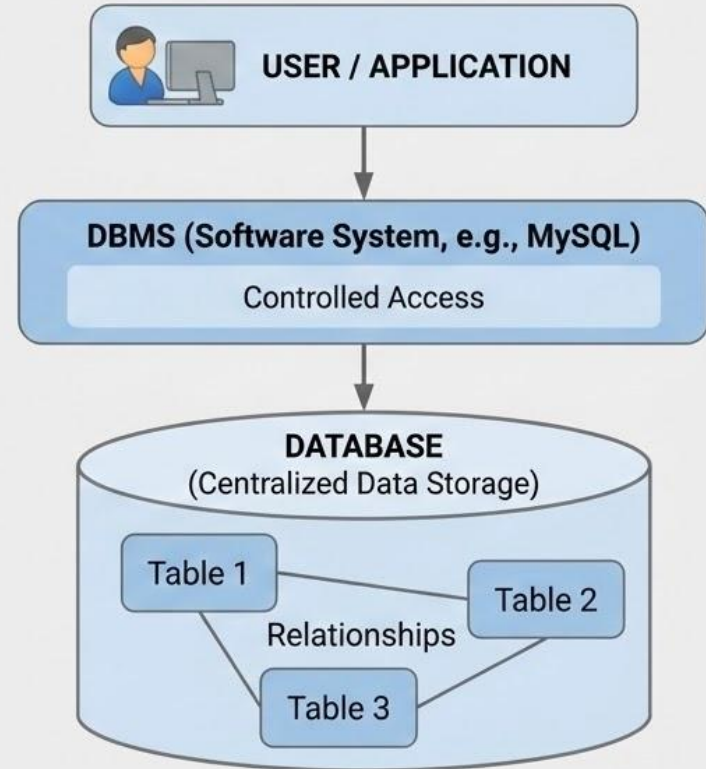
- Limitations:
 - Concurrent Access Issues: What if 2 people edit the same file simultaneously?
 - Data corruption
 - Loss of updates
 - No Backup/Recovery: Corrupted file = data loss
 - Scalability: 10GB+ of files = slow, inefficient searching

File System vs. DBMS

- DBMS
- A database management system (DBMS) is a way to interact with a database using a graphical interface or textual language.
- Examples of DBMS: MySQL, Oracle, PostgreSQL, SQLite, Microsoft Access, MongoDB etc.
- How it works:
- Centralized data storage
- Organized in tables with relationships
- Controlled access through a software system (MySQL)

How DBMS Works

- Centralized data storage
- Organized in tables with relationships
- Controlled access through a software system (MySQL)



Advantages of DBMS

- Reduced Redundancy: ACID properties and normalization eliminate duplicate data
- Fast Retrieval: Indexes, optimized queries → results in milliseconds
- Concurrent Access: Multiple users can access simultaneously without conflicts (locking mechanisms)
- Security: User permissions, roles, encryption
- Backup & Recovery: Built-in tools for backup and restoration
- Scalability: Can handle millions of records efficiently
- Data Integrity: Constraints ensure only valid data is stored

DBMS Components

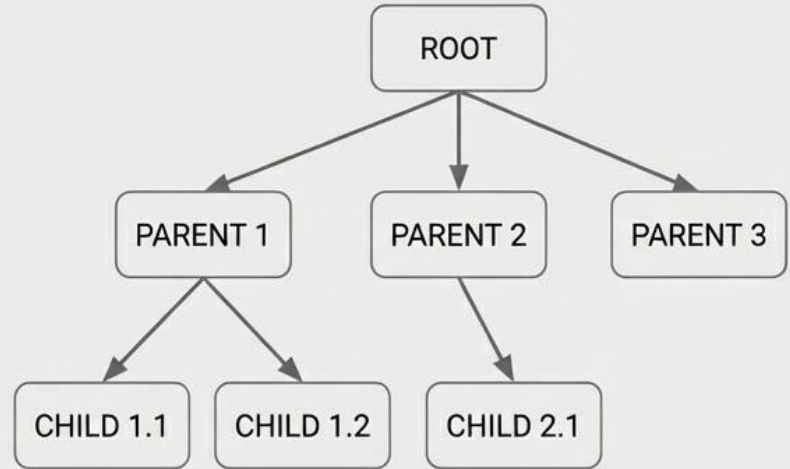
- Query Engine: Processes SQL queries
- Query Optimizer: Decides the most efficient way to execute a query
- Transaction Manager: Ensures ACID properties (Atomicity, Consistency, Isolation, Durability)
- Buffer Manager: Manages memory cache for fast access
- Concurrency Control: Handles multiple simultaneous users
- Recovery Manager: Backup and restoration
- Security Manager: User authentication and authorization

Think of it as: A librarian who knows exactly where every book is, can retrieve it instantly, ensures only authorized people access it, and keeps backups of everything.

DBMS TYPES

Hierarchical DBMS (1960s-1970s)

- Data organized in a tree-like structure with parent-child relationships.
- Each child record has only one parent record (1:N relationship).
- Efficient for specific, predefined queries but inflexible for ad-hoc queries.
- Example: An organizational chart or a file system.

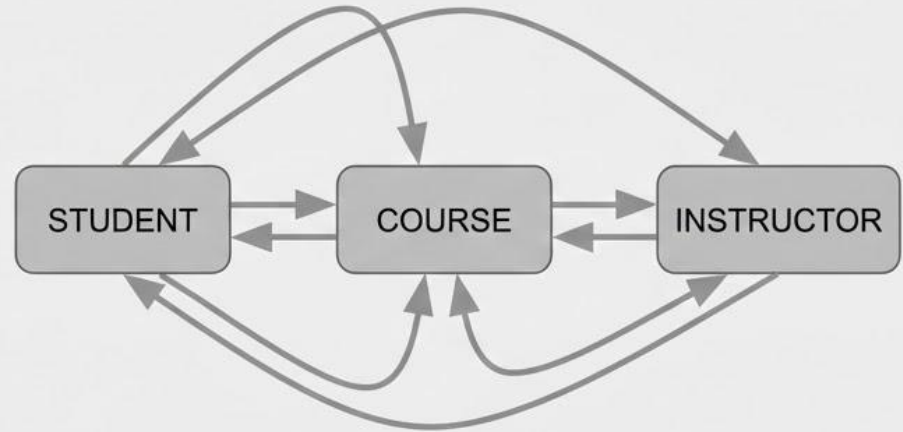


Limitations

- Inflexible (can't query across different branches easily)
- Redundant data (each department stores its own employee list)
- Hard to maintain
- Access data by following the hierarchy only.
- When used: Early banking systems, inventory management

Network DBMS (1970s-1980s)

- Graph-like with multiple relationships.
- Student can enroll in multiple Courses
- Course can have multiple Students
- Instructor can teach multiple Courses



Network DBMS

- How it works:
 - Allows many-to-many relationships
 - More flexible than hierarchical
 - Uses pointers to navigate between records
 - Example: CODASYL databases
- Limitations:
 - Complex to query
 - Hard to modify structure
 - Steep learning curve
- When used: Early reservation systems, complex relationships

Relational DBMS (1980s-Present)

Structure: Tables with rows and columns.

Students Table

ID	Name	DeptID
1	Raj	101
2	Priya	102
3	Arjun	101

Departments Table

DeptID	Name
101	CSE
102	ECE
103	Mech

How it works:

- Simple, intuitive structure
- Tables are related through primary and foreign keys
- Uses SQL (Structured Query Language)

Examples:

- MySQL
- PostgreSQL
- Oracle
- SQL Server
- MariaDB

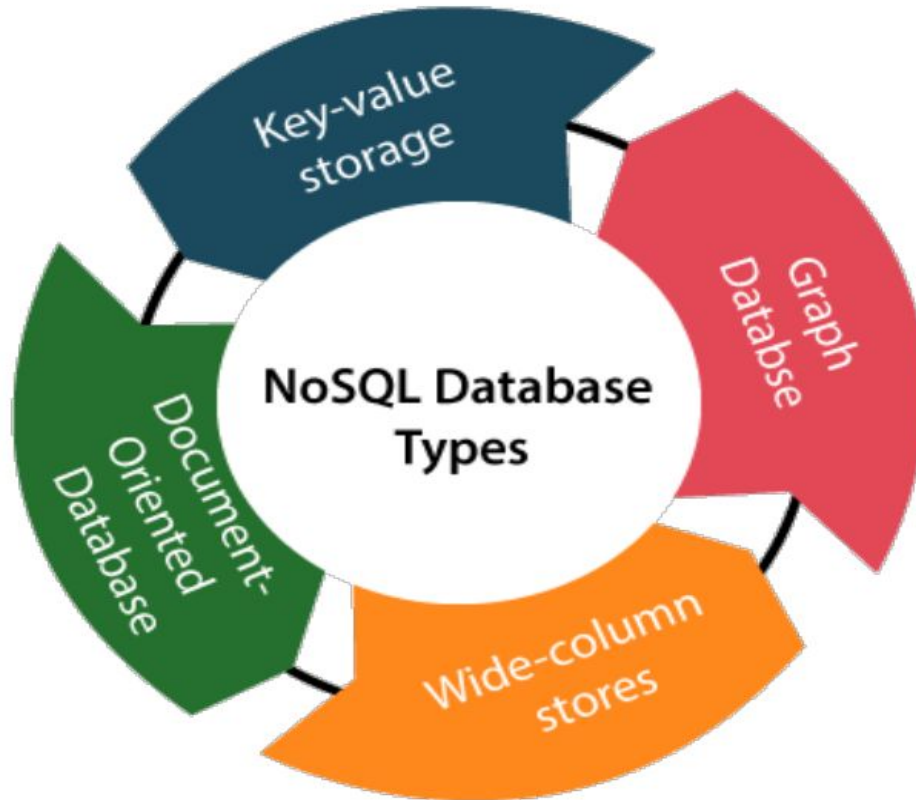
Relational Database: Advantages

- Easy to understand and design
- Flexible querying
- ACID compliance -> Atomicity, Consistency, Isolation, Durability
 - They ensure your database transactions are safe, correct, independent, and permanent.
- Scalable
- When used: 90% of business applications today.

NO-SQL DBMS

- Non-SQL/Not Only SQL is a type of database that is used for storing a wide range of data sets.
- It is not a relational database as it stores data not only in tabular form but in several different ways.
- It came into existence when the demand for building modern applications increased. Thus, NoSQL presented a wide variety of database technologies in response to the demands.
- We can further divide a NoSQL database into the following four types

NO-SQL DBMS



Feature	Relational (RDBMS)	NoSQL
Schema	Rigid, predefined	Flexible, dynamic
Scaling	Vertical (bigger server)	Horizontal (more servers)
Consistency	ACID compliant	Eventually consistent
Querying	SQL (powerful)	API/Query language varies
Relationships	Easy with foreign keys	Complex, stored as nested data
Transactions	Multi-row transactions	Limited transaction support
Speed	Good (optimized for complex queries)	Faster for simple reads
Best For	Structured data, business logic	Unstructured data, big scale

What is SQL?

- SQL is the standard language for dealing with Relational Databases.
- SQL can be used to insert, search, update, and delete database records. SQL can do lots of other operations, including optimizing and maintenance of databases.
- SQL Full Form - SQL stands for Structured Query language, pronounced as "S-Q-L" or sometimes as "See-Quel"... Relational databases like MySQL Database, Oracle, MS SQL Server, Sybase, etc. use ANSI SQL.

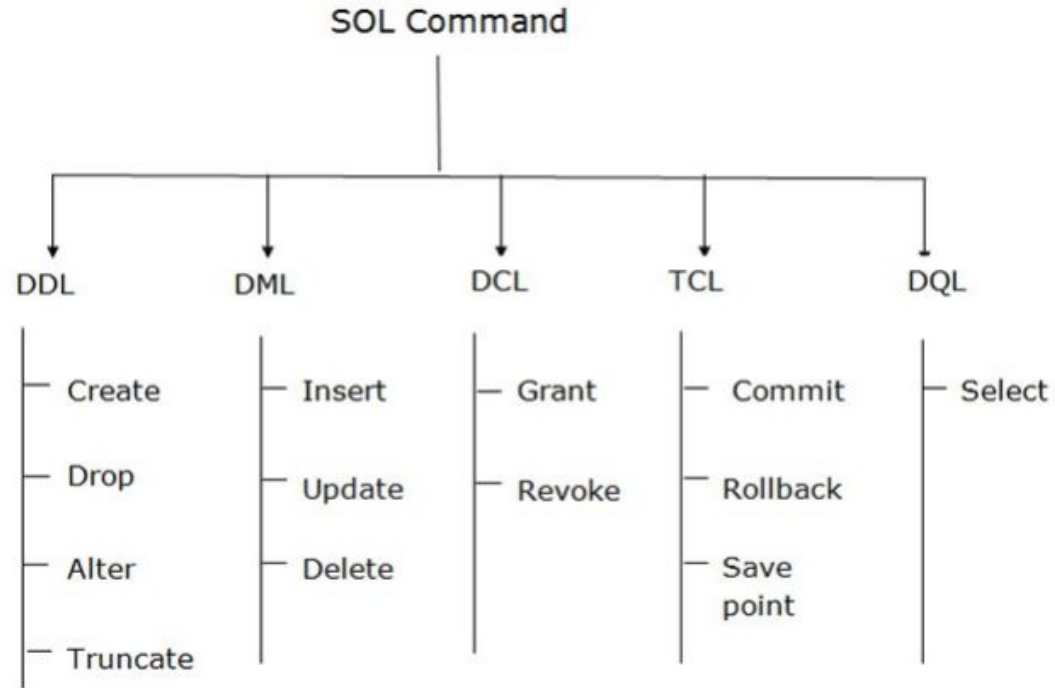
MYSQL(RDBMS)

- MySQL is released under an open-source license. So you have nothing to pay to use it.
- MySQL is a very powerful program in its own right. It handles a large subset of the functionality of the most expensive and powerful database packages.
- MySQL uses a standard form of the well-known SQL data language.
- MySQL works on many operating systems and with many languages including Python, PHP, PERL, C, C++, JAVA, etc. • MySQL works very quickly and works well even with large data sets.

Types of SQL Statements

- Data Definition Language (DDL)
- Data Manipulation Language (DML)
- Data Control Language (DCL)
- Transaction Control Language (TCL)
- Data Query Language (DQL)

Types of SQL Statements



Terminologies

- Database – A database is a collection of tables, with related data.
- Table – A table is a matrix with data. A table in a database looks like a simple spreadsheet.
- Column – One column (data element) contains data of one and the same kind, for example the column postcode.
- Row – A row (= tuple, entry or record) is a group of related data, for example the data of one subscription.
- Redundancy – Storing data twice, redundantly to make the system faster.

Terminologies

- Primary Key - A primary key is unique. A key value can not occur twice in one table. With a key, you can only find one row.
- Foreign Key - A foreign key is the linking pin between two tables.
- Compound Key - A compound key (composite key) is a key that consists of multiple columns, because one column is not sufficiently unique.
- Unique Key - No two rows can have the same value in this field (NULL is allowed)
- Index - An index in a database resembles an index at the back of a book.
- Referential Integrity - Referential Integrity makes sure that a foreign key value always points to an existing row.

Data Types

- A Data Type specifies a particular type of data, like integer, floating points, Boolean, etc.
- It also identifies the possible values for that type, the operations that can be performed on that type, and the way the values of that type are stored.
- In MySQL, each database table has many columns and contains specific data types for each column

Data Types

- We can determine the data type in MySQL with the following characteristics
 - The type of values (fixed or variable) it represents.
 - The storage space it takes is based on whether the values are a fixed-length or variable length.
 - Its values can be indexed or not.
 - How MySQL performs a comparison of values of a particular data type.

Data Types: Numeric

- MySQL has all essential SQL numeric data types. These data types can include the exact numeric data types (For example, integer, decimal, numeric, etc.), as well as the approximate numeric data types (For example, float, real, and double precision).
- It also supports BIT datatype to store bit values. In MySQL, numeric data types are categorized into two types, either signed or unsigned except for bit data type.

Data Types: Numeric

- **TINYINT**
 - It is a very small integer that can be signed or unsigned. If signed, the allowable range is from -128 to 127. If unsigned, the allowable range is from 0 to 255. We can specify a width of up to 4 digits. It takes 1 byte for storage.
- **SMALLINT**
 - It is a small integer that can be signed or unsigned. If signed, the allowable range is from -32768 to 32767. If unsigned, the allowable range is from 0 to 65535. We can specify a width of up to 5 digits. It requires 2 bytes for storage.

Data Types: Numeric

- Similarly you have Mediumint, int, bigint datatypes as well.
- FLOAT(m,d) - It is a floating-point number that cannot be unsigned. You can define the display length (m) and the number of decimals (d). This is not required and will default to 10,2, where 2 is the number of decimals, and 10 is the total number of digits (including decimals). Decimal precision can go to 24 places for a float type. It requires 2 bytes for storage.

Data Types: String

- `CHAR(SIZE)` - It can have a maximum size of 255 characters. Here size is the number of characters to store. Fixed-length strings. Space padded on the right to equal size characters.
- `VARCHAR(size)` - It can have a maximum size of 255 characters. Here size is the number of characters to store. Variable-length string.

Data Definition Language (DDL)

- DDL changes the structure of the table like creating a table, deleting a table, altering a table, etc.
- All the command of DDL are auto-committed that means it permanently save all the changes in the database.
- Here are some commands that come under DDL:
 - CREATE
 - ALTER
 - DROP
 - TRUNCATE

Data Definition Language (DDL)

- **CREATE**
 - It is used to create a new table in the database.
- **DROP**
 - It is used to delete both the structure and record stored in the table.
- **ALTER:**
 - It is used to alter the structure of the database. This change could be either to modify the characteristics of an existing attribute or probably to add a new attribute.
- **TRUNCATE**
 - It is used to delete all the rows from the table and free the space containing the table.

CREATE

- It is used to create new entities in the database.
- It can be used to create a table, indexes, views or even database itself

CREATE DATABASE CollegeDB;

Creating a table

- When you create the table there are multiple things you have to specify along with the column names.

```
CREATE TABLE Students (  
    StudentID INT NOT NULL,  
    Name VARCHAR(100) NOT NULL,  
    Email VARCHAR(100)  
);
```

ALTER – Modifying Table Structure

- After creating a table, you realize:
 - You forgot to add a column (e.g., 'Phone')
 - A column name is misspelled
 - You need to change data type
 - You need to add/remove constraints
- You can add all these using ALTER

ALTER – Modifying Table Structure

- You created the Students table but forgot to add a "Phone" column!

```
ALTER TABLE Students  
ADD COLUMN Phone VARCHAR(10);
```

- Adding with constraints

```
ALTER TABLE Students  
ADD COLUMN Address VARCHAR(200) NOT NULL  
DEFAULT 'Not Provided';
```

ALTER – Modifying Table Structure

- Modifying column (changing its properties):

```
ALTER TABLE Students  
MODIFY COLUMN Phone VARCHAR(15);  
(Changed from 10 to 15 characters)
```

- Renaming a column:

```
ALTER TABLE Students  
CHANGE COLUMN Phone PhoneNumber VARCHAR(10);  
(Renamed from "Phone" to "PhoneNumber")
```

ALTER – Modifying Table Structure

- Dropping a column:

```
ALTER TABLE Students  
DROP COLUMN Address;
```

- Adding a PRIMARY KEY (if table doesn't have one):

```
ALTER TABLE Students  
ADD PRIMARY KEY (StudentID);
```

DROP vs. TRUNCATE – When You Want to Delete

- **DROP TABLE**

- Purpose: Delete the entire table structure + all data
- Effect:
 - Table is completely removed
 - All data is deleted
 - Table structure is gone
 - Cannot be recovered (unless you have backups)
 - Space is released back to the system

DROP vs. TRUNCATE – When You Want to Delete

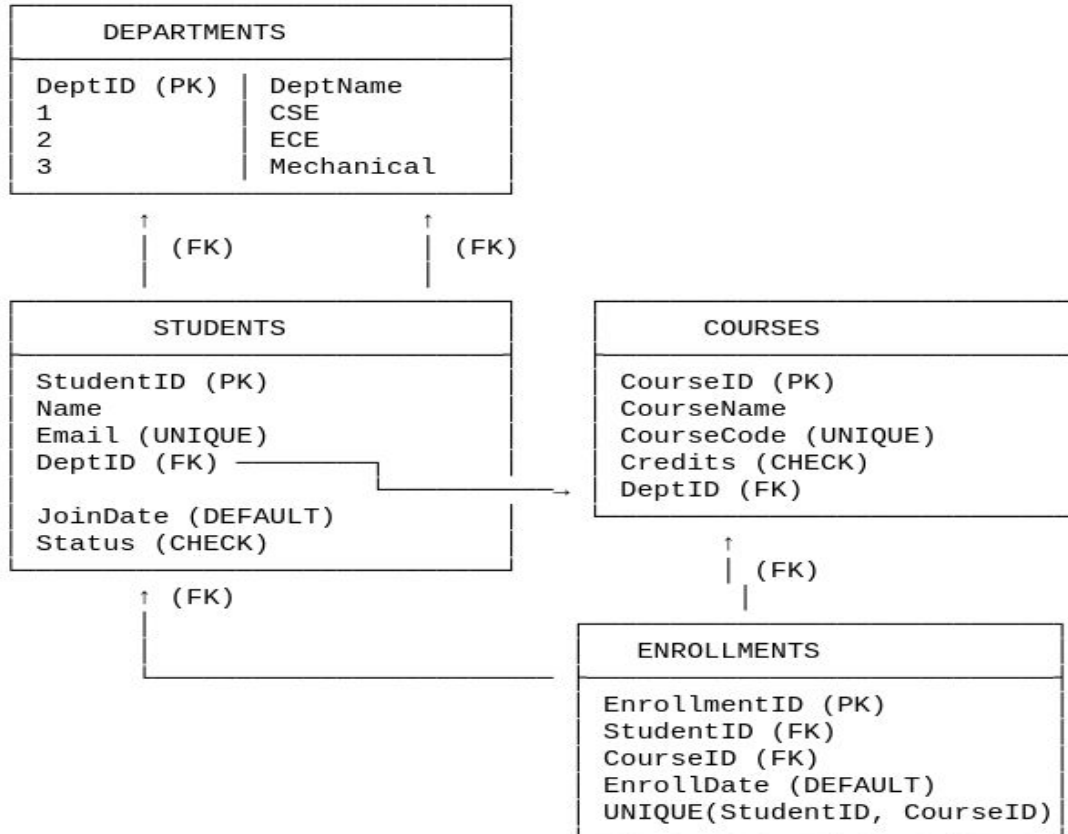
- When to use:
 - You created a table by mistake
 - You're redesigning the entire system
 - Table is no longer needed

Danger level: ●●● Very dangerous! (but useful for cleanup during development)

DROP vs. TRUNCATE – When You Want to Delete

- TRUNCATE
 - Purpose: Delete all data from table, but keep the structure
- Effect:
 - Table structure remains (columns, constraints, etc.)
 - All data is deleted
 - Cannot be recovered easily (though better than DROP)
 - Space is released

TRY IT OUT...



Data Manipulation (DML)

- DML commands are used to work with the actual data in your tables.
- Following are the 3 main DML commands
 - INSERT: Add new rows to table
 - UPDATE: Modify existing data
 - DELETE: Remove rows

Data Query Language (DQL)

- Here only one command is there => SELECT
- You'll use SELECT constantly!
- SELECT is how you READ data from tables.

```
SELECT column1, column2, ... FROM table_name;
```

```
SELECT * FROM table_name;
```

WHERE – The Filter Clause

- WHERE filters rows based on conditions.

Syntax ->

SELECT columns FROM table WHERE condition;

- Combine conditions for complex filtering:
 - AND - Both conditions must be true
 - OR - At least one condition must be true
 - NOT - Opposite of condition
 - Complex: AND + OR
- ```
SELECT * FROM Students
WHERE (marks > 80 AND department = 'CSE') OR city
= 'Mumbai';
```

# WHERE with BETWEEN, IN, LIKE

- BETWEEN - Range of values  
SELECT \* FROM Students WHERE marks BETWEEN 80 AND 90;
- IN - Match any value in list  
SELECT \* FROM Students WHERE department IN ('CSE', 'ECE');
- LIKE - Pattern matching
  - LIKE 'A%' = Starts with A
  - LIKE '%a' = Ends with a
  - LIKE '%ar%' = Contains 'ar'
  - LIKE 'R\_j' = R, any 1 char, j

# UPDATE – Modifying Data

- UPDATE changes existing data in rows.
- Syntax:  
UPDATE table\_name  
SET column1 = value1, column2 = value2  
WHERE condition;
-  **CRITICAL:** Always use WHERE! Without WHERE, ALL rows are updated!

## UPDATE – Examples

- Example 1: Update one student  
UPDATE Students  
SET marks = 95  
WHERE student\_id = 1;
- Example 2: Update multiple columns  
UPDATE Students  
SET marks = 90, city = 'Hyderabad'  
WHERE student\_id = 2;

## UPDATE – Examples

- Example 3: Update multiple rows  
UPDATE Students  
SET marks = marks + 5  
WHERE department = 'CSE';
- Update without WHERE (DANGEROUS!)  
UPDATE Students SET marks = 0;

## DELETE – Removing Data

- DELETE removes rows from table.  
DELETE FROM table\_name WHERE condition;
- Again: WHERE is CRITICAL!
- Delete ALL rows (DANGEROUS!)  
DELETE FROM Students;

# **SORTING, AGGREGATION & TRANSACTIONS**

## ORDER BY – Sorting Results

- ORDER BY sorts query results.

```
SELECT * FROM table_name ORDER BY column
ASC|DESC;
```

```
SELECT * FROM Students ORDER BY marks DESC;
```

- Sort by multiple columns

```
SELECT * FROM Students ORDER BY department
ASC, marks DESC;
```

## DISTINCT – Removing Duplicates

- DISTINCT shows unique values (removes duplicates).  
`SELECT DISTINCT column FROM table_name;`
- Example => Find all unique departments  
`SELECT DISTINCT column FROM table_name;`
- Example=>Count unique departments  
`SELECT COUNT(DISTINCT department) FROM Students;`

# Aggregate Functions – Summary Statistics

- Aggregate functions calculate across multiple rows.

| Function | Purpose    | Example                                                 |
|----------|------------|---------------------------------------------------------|
| COUNT()  | Count rows | <code>COUNT(*)</code> or <code>COUNT(student_id)</code> |
| SUM()    | Sum values | <code>SUM(marks)</code>                                 |
| AVG()    | Average    | <code>AVG(marks)</code>                                 |
| MIN()    | Minimum    | <code>MIN(marks)</code>                                 |
| MAX()    | Maximum    | <code>MAX(marks)</code>                                 |

## GROUP BY – Grouping Data

- *GROUP BY* divides rows into groups and applies aggregates to each group.

```
SELECT column, AGGREGATE_FUNCTION(column)
FROM table_name
GROUP BY column;
```

# GROUP BY – Grouping Data

- Example 1: Average marks per department

```
SELECT department, AVG(marks)
FROM Students
GROUP BY department;
```

- How it works:
  - Divides Students into 4 groups (one per department)
  - Calculates *AVG(marks)* for each group

## HAVING – Filtering Groups

- HAVING filters groups (like WHERE filters rows).
- WHERE filters rows before grouping
- HAVING filters groups after grouping

```
SELECT column, AGGREGATE
FROM table_name
GROUP BY column
HAVING condition;
```

## HAVING – Filtering Groups

- Example 1: Departments with average marks  $> 85 \Rightarrow$

```
SELECT department, AVG(marks) AS AvgMarks
FROM Students
GROUP BY department
HAVING AVG(marks) > 85;
```

# Transactions – COMMIT & ROLLBACK

- Transactions ensure data integrity.
- Concept: A group of SQL operations that either all succeed or all fail.
- Analogy: Think of a bank transfer
  - Move \$100 from Account A to Account B
  - If it succeeds: Both operations complete (debit + credit)
  - If it fails: Both are undone (ROLLBACK) to prevent loss

# Transactions – COMMIT & ROLLBACK

- COMMIT: Save changes permanently
- ROLLBACK: Undo all changes since last COMMIT
- Why Transactions Matter:-  
Scenario: Instructor updates student marks

UPDATE Students SET marks = 95 WHERE student\_id = 1;

UPDATE Students SET marks = 85 WHERE student\_id = 2;

What if power goes out in middle?

Raj's marks updated to 95

Priya's marks not updated yet

Data is inconsistent!

# Transactions – COMMIT & ROLLBACK

- Solution: Use transaction

START TRANSACTION; -- Begin transaction

UPDATE Students SET marks = 95 WHERE student\_id = 1;  
UPDATE Students SET marks = 85 WHERE student\_id = 2;

COMMIT; -- Save all changes permanently

- Now both happen together or neither happens.

# COMMIT vs ROLLBACK – Example

- Scenario: Wrong update mistake!
  - Step 1: Begin  
START TRANSACTION;
  - Step 2: Wrong update! Reduced everyone's marks  
UPDATE Students SET marks = 40;
  - Step 3: Oops! I made a mistake. UNDO it!  
ROLLBACK;
  - Step 4: All marks are back to original (before update)  
SELECT \* FROM Students;

# COMMIT vs ROLLBACK – Example

- process:-

Before transaction: Raj=85, Priya=92



UPDATE SET marks = 40



After update (before COMMIT): Raj=40, Priya=40



ROLLBACK (undo)



Back to: Raj=85, Priya=92 (Safe!)

# COMMIT vs ROLLBACK – Example

- example: Correct update

START TRANSACTION;

UPDATE Students SET marks = 95 WHERE student\_id = 1;

UPDATE Students SET marks = 90 WHERE student\_id = 2;

COMMIT; -- Save permanently

-- Now these changes are permanent (can't rollback)

# AUTO\_COMMIT Mode

- Important: MySQL has `AUTO_COMMIT` enabled by default!
- What it means:
  - Every statement auto-commits immediately
  - No need to explicitly `COMMIT`
  - Changes are permanent right away
- This is why `ROLLBACK` doesn't work in normal mode:  
`UPDATE Students SET marks = 40; -- Auto-committed immediately`  
`ROLLBACK; -- Too late! Already committed, can't undo`

# To use manual transaction mode:

SET autocommit = 0; -- Turn OFF auto-commit

UPDATE Students SET marks = 40;  
-- Changes are NOT permanent yet

ROLLBACK; -- Undo the change (works!)

-- Or

COMMIT; -- Make changes permanent

## To use manual transaction mode:

- Or explicitly start transaction:

`START TRANSACTION; -- Disables auto-commit for this transaction`

`UPDATE Students SET marks = 40;`  
`ROLLBACK; -- Works!`

`-- After transaction, auto-commit is back on`

# ADVANCED LOGIC- JOINS

## PL/SQL

# What is Database Normalization?

- Normalization is a process of organizing data in a database to:
  - Minimize data redundancy (no duplication)
  - Improve data integrity (consistency)
  - Eliminate insertion/update/deletion anomalies

# Normal Forms – The Normalization Ladder

Start: Unnormalized Data (MESSY)



1NF (First Normal Form) - Atomic values, no repeating groups



2NF (Second Normal Form) - Remove partial dependencies



3NF (Third Normal Form) - Remove transitive dependencies



BCNF (Boyce-Codd Normal Form) - Super strict (rarely needed)

Most real-world databases aim for 3NF. Beyond that is overkill for most applications.

# Unnormalized Data – The Problem

Bad Design (All in one table)

StudentCourseInfo Table:

| StudentID | Name  | Dept | CourseID | CourseName | Marks | Credits |
|-----------|-------|------|----------|------------|-------|---------|
| 1         | Raj   | CSE  | 101      | Python     | 85    | 4       |
| 1         | Raj   | CSE  | 102      | Java       | 92    | 3       |
| 1         | Raj   | CSE  | 103      | SQL        | 88    | 4       |
| 2         | Priya | ECE  | 101      | Python     | 78    | 4       |
| 2         | Priya | ECE  | 104      | C++        | 82    | 3       |

# Unnormalized Data – The Problem

- Problems (Anomalies):
  - Redundancy: "Raj" and "CSE" repeated 3 times
    - Takes more storage
    - If Raj changes dept, update 3 rows!
  - Insert Anomaly: Can't add a new student until they enroll in a course
    - Want to add: StudentID=10, Name=Arjun, Dept=MECH But CourseID is mandatory in this design! Can't insert until they enroll in a course.

# Unnormalized Data – The Problem

- Problems (Anomalies):
  - Update Anomaly: Update is inconsistent
    - Change Raj's dept from CSE to MECH
    - Update row 1 ✓ Updated
    - Update row 2 ✓ Updated
    - Forgot to update row 3! ✗ Inconsistent
    - Now: Is Raj in CSE or MECH? Database doesn't know!

# 1NF (First Normal Form) – Atomic Values

- Rule: Each cell must contain only ONE value (no lists, arrays, repeating groups).
- Bad (Violates 1NF): Courses column has multiple values in one cell!

Student Table:

| StudentID | Name  | Courses                   |
|-----------|-------|---------------------------|
| 1         | Raj   | Python, Java, SQL (LIST!) |
| 2         | Priya | Python, C++ (LIST!)       |

# 1NF (First Normal Form) – Atomic Values

- *Good (1NF): (Each cell has ONE value)*

Students Table:

| StudentID | Name  |
|-----------|-------|
| 1         | Raj   |
| 2         | Priya |

StudentCourses Table (Separate):

| StudentID | CourseID |
|-----------|----------|
| 1         | 101      |
| 1         | 102      |
| 1         | 103      |
| 2         | 101      |
| 2         | 104      |

## 2NF (Second Normal Form)

- Rule:- Remove "partial dependencies"
  - Must be in 1NF (already is)
  - All non-key columns must depend on the entire primary key (not just part of it)

# 2NF (Second Normal Form)

- Violates 2NF

Enrollments Table (composite PK: StudentID + CourseID):

| StudentID | CourseID | StudentName | Marks |
|-----------|----------|-------------|-------|
| 1         | 101      | Raj         | 85    |
| 1         | 102      | Raj         | 92    |
| 2         | 101      | Priya       | 78    |

Problem:

- PK is (StudentID, CourseID) = composite
- But StudentName depends ONLY on StudentID (not on CourseID)
- If we know StudentID=1, we know StudentName=Raj (don't need CourseID)
- This is PARTIAL dependency

Students Table:

| StudentID | Name  |
|-----------|-------|
| 1         | Raj   |
| 2         | Priya |

(StudentName depends on StudentID only ✓)

Enrollments Table:

| StudentID | CourseID | Marks |
|-----------|----------|-------|
| 1         | 101      | 85    |
| 1         | 102      | 92    |
| 2         | 101      | 78    |

(All columns depend on BOTH StudentID AND CourseID ✓)



**The Fix:  
Separate  
Tables**

## 3NF (Third Normal Form)

- Rule:- Remove dependencies between non-key columns
  - Must be in 2NF (already is)
  - No "transitive dependencies" (non-key column depends on another non-key column)

Students Table:

| StudentID | Name  | DeptID | DeptName     |
|-----------|-------|--------|--------------|
| 1         | Raj   | 101    | Computer Sci |
| 2         | Priya | 102    | Electronics  |
| 3         | Arjun | 101    | Computer Sci |

Problem:

- DeptName depends on DeptID (transitive dependency)
- StudentID  $\rightarrow$  DeptID  $\rightarrow$  DeptName
- DeptName is stored in Students table, but it's not unique per student
- "Computer Sci" appears twice (redundancy!)

**Violates  
3NF**

Students Table:

| StudentID | Name  | DeptID |
|-----------|-------|--------|
| 1         | Raj   | 101    |
| 2         | Priya | 102    |
| 3         | Arjun | 101    |

(Only StudentID, Name, DeptID - no DeptName)

Departments Table:

| DeptID | DeptName         |
|--------|------------------|
| 101    | Computer Science |
| 102    | Electronics      |

(DeptName stored once per department ✓)

**The Fix:**  
Removes  
redundancy,  
one source  
of truth for  
each fact.

# Introduction to JOINS

- After normalization, your data is in multiple tables.  
How do you combine them?
- Answer: JOINS!
- Example=>

You want: "Show me StudentName, CourseName, and Marks for StudentID=1"

This data is in THREE tables:

- Students (has StudentName)
- Courses (has CourseName)
- Enrollments (has Marks, StudentID, CourseID)

## Types of JOINS:

- INNER JOIN (only matching records)
- LEFT JOIN (all from left table + matching from right)
- RIGHT JOIN (all from right table + matching from left)
- CROSS JOIN (Cartesian product, rarely used)

# INNER JOIN – Matching Records Only

- Return rows where there's a match in BOTH tables

Students Table:

| ID | Name |
|----|------|
|----|------|

|   |       |
|---|-------|
| 1 | Raj   |
| 2 | Priya |
| 3 | Arjun |

Enrollments Table:

| StudentID | CourseID | Marks |
|-----------|----------|-------|
|-----------|----------|-------|

|   |     |    |
|---|-----|----|
| 1 | 101 | 85 |
| 1 | 102 | 92 |
| 2 | 101 | 78 |
| 4 | 103 | 95 |

← StudentID 4 doesn't exist!

INNER JOIN Result:

| ID | Name | StudentID | CourseID | Marks |
|----|------|-----------|----------|-------|
|----|------|-----------|----------|-------|

|   |       |   |     |    |
|---|-------|---|-----|----|
| 1 | Raj   | 1 | 101 | 85 |
| 1 | Raj   | 1 | 102 | 92 |
| 2 | Priya | 2 | 101 | 78 |

# INNER JOIN – Matching Records Only

- Syntax

```
SELECT
 s.student_id,
 s.name,
 e.course_id,
 e.marks
FROM Students s
INNER JOIN Enrollments e
 ON s.student_id = e.student_id;
```

# LEFT JOIN – All Left + Matching Right

- Return ALL rows from left table. If no match in right,.

show NULL

Students Table:

| ID | Name  |
|----|-------|
| 1  | Raj   |
| 2  | Priya |
| 3  | Arjun |

Enrollments Table:

| StudentID | CourseID | Marks |
|-----------|----------|-------|
| 1         | 101      | 85    |
| 1         | 102      | 92    |
| 2         | 101      | 78    |
| 4         | 103      | 95    |

LEFT JOIN Result:

| ID | Name  | StudentID | CourseID | Marks |
|----|-------|-----------|----------|-------|
| 1  | Raj   | 1         | 101      | 85    |
| 1  | Raj   | 1         | 102      | 92    |
| 2  | Priya | 2         | 101      | 78    |
| 3  | Arjun | NULL      | NULL     | NULL  |

← Arjun has no enrollments!

# LEFT JOIN – All Left + Matching Right

- Syntax

```
SELECT
 s.student_id,
 s.name,
 e.course_id,
 e.marks
FROM Students s
LEFT JOIN Enrollments e
 ON s.student_id = e.student_id;
```

# RIGHT JOIN – All Right + Matching Left

- Return ALL rows from right table. If no match in left,

show NULL

Students Table:

ID|Name

1 |Raj

2 |Priya

3 |Arjun

Enrollments Table:

StudentID|CourseID|Marks

1 |101 |85

1 |102 |92

2 |101 |78

4 |103 |95

RIGHT JOIN Result:

ID|Name |StudentID|CourseID|Marks

1 |Raj |1 |101 |85

1 |Raj |1 |102 |92

2 |Priya |2 |101 |78

NULL|NULL |4 |103 |95 ← StudentID 4 doesn't exist in

# CROSS JOIN – Cartesian Product

- Combine EVERY row from left table with EVERY row from right table
- Rarely! Maybe for generating all possible combinations

Students Table:

| ID | Name  |
|----|-------|
| 1  | Raj   |
| 2  | Priya |

Courses Table:

| CourseID | Name   |
|----------|--------|
| 101      | Python |
| 102      | Java   |

CROSS JOIN Result:

| StudentID | StudentName | CourseID | CourseName |
|-----------|-------------|----------|------------|
| 1         | Raj         | 101      | Python     |
| 1         | Raj         | 102      | Java       |
| 2         | Priya       | 101      | Python     |
| 2         | Priya       | 102      | Java       |

(2 students × 2 courses = 4 combinations)

## Joining 3+ Tables

```
SELECT
 s.student_id,
 s.name,
 c.course_name,
 e.marks
FROM Students s
INNER JOIN Enrollments e ON s.student_id =
e.student_id
INNER JOIN Courses c ON e.course_id = c.course_id;
```

# JOIN Conditions with WHERE

-- Show marks > 80 for all students in their courses

```
SELECT
 s.student_id,
 s.name,
 c.course_name,
 e.marks
FROM Students s
INNER JOIN Enrollments e ON s.student_id = e.student_id
INNER JOIN Courses c ON e.course_id = c.course_id
WHERE e.marks > 80; -- Filter AFTER joining
```

# PYTHON INTEGRATION

# Why Python + MySQL?

- Why Connect?
  - Database is "brain" (stores data)
  - Python is "hands" (does work, presents data)
  - Together: Complete application
- Real-world examples:
  - Facebook: Python backend → MySQL database
  - Airbnb: Python + Node.js → PostgreSQL/MySQL
  - Instagram (originally): Python + MySQL
  - Your college system: Python app → College database

# Implementation Flow

User clicks "Enroll Student" button



Python app processes the click



Python connects to MySQL database



Executes enrollment stored procedure



Trigger automatically logs the action



Python fetches updated enrollment list



Displays results to user

# Connection String

- Connection - Telling Python where to find your database and how to log in.
- pieces of info needed:
  - Host: Where is the database? (localhost = same computer, 127.0.0.1 = IP)
  - User: Username to login (usually 'root' for practice, 'app\_user' for production)
  - Password: Database password (you set this when installing)
  - Database: Which database?

`mysql://root:password@localhost:3306/Collegedb`

# Creating A New User

- For this we will create a new user and will give it the access of our database

```
-- create a normal user for your Python code
CREATE USER 'pyuser'@'localhost' IDENTIFIED BY 'PyUser@123';

-- give it full rights on your training DB
GRANT ALL PRIVILEGES ON collegedb.* TO 'pyuser'@'localhost';

FLUSH PRIVILEGES;
EXIT;
```